Algorytmy macierzowe

Sprawozdanie z zadania 1

Adrian Mikoda Szymon Płókarz

25.10.2026 r.

Spis treści

1. Dane techniczne . . . 2
2. Treść zadań . . . 2
   2.1. Zadania implementacyjne . . . 2
   2.2. Wymagania dotyczące raportu . . . 2
   2.3. Weryfikacja poprawności wyników . . . 3
3. Realizacja ćwiczenia . . . 3
   3.1. Środowisko implementacji . . . 3
   3.2. Generowanie macierzy i zliczanie operacji . . . 3
   3.3. Punkt 1 – rekurencyjne mnożenie macierzy metodą Binét'a . . . 4
   3.4. Punkt 2 – rekurencyjne mnożenie macierzy metodą Strassena . . . 5
   3.5. Punkt 3 – rekurencyjne mnożenie macierzy metodą Ladermana . . . 6
   3.6. Pomiar czasu i weryfikacja poprawności . . . 7
4. Wyniki . . . 15
   4.1. Czas działania . . . 15
   4.2. Liczba operacji zmiennoprzecinkowych . . . 15
5. Złożoność obliczeniowa . . . 18
   5.1. Analiza teoretyczna . . . 18
   5.2. Weryfikacja eksperymentalna . . . 18
6. Wnioski . . . 19

1. Dane techniczne

- System operacyjny: `Windows Subsystem for Linux Distributions: Ubuntu-26.04`
- Procesor: `AMD Ryzen 5 7530U (2.0 - 4.5 GHz)`
- Pamięć RAM: `16GB LPDDR4 (2133 MHz)`
- Język programowania: `C++ w standardzie C++17`
- Kompilator: `g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0`
- Wizualizacja wyników: `Python 3.13.14`
- Użyte biblioteki: `matplotlib 3.11.2, numpy 2.5.4`

2. Treść zadań

2.1. Zadania implementacyjne

W ramach pierwszego zestawu zadań, realizowanego w parach dwuosobowych, należy wybrać dowolny język programowania, który nie jest językiem interpretowanym.

Należy wygenerować losowe macierze kwadratowe o elementach należących do przedziału otwartego $(0.00000001, 1.0)$, a następnie zaimplementować następujące algorytmy:

1. Rekurencyjne mnożenie macierzy metodą Binét'a.
2. Rekurencyjne mnożenie macierzy metodą Strassena.
3. Rekurencyjne mnożenie macierzy metodą Ladermana.

Podczas wykonywania mnożenia macierzy należy zliczać liczbę operacji zmiennoprzecinkowych, tj. dodawania, odejmowania, mnożenia i dzielenia ($+, -, \times, \div$).

Wszystkie algorytmy należy zaprojektować i zbadać dla macierzy kwadratowych o dowolnym rozmiarze, tj. o jednakowej liczbie wierszy i kolumn.

2.2. Wymagania dotyczące raportu

W ramach sprawozdania należy przygotować następujące elementy:

1. **Opis pseudokodu** rekurencyjnego algorytmu mnożenia macierzy – 1 punkt.
2. **Najważniejsze fragmenty kodu źródłowego** przedstawiające implementację algorytmu – 1 punkt.
3. **Charakterystyka środowiska wykonawczego** – uzasadnienie, dlaczego wybrany język programowania nie jest interpretowany, oraz podanie informacji o procesorze i pamięci komputera, na którym przeprowadzono eksperymenty – 1 punkt.
4. **Wykres czasu działania algorytmu** – 2 punkty.
   - Oś pozioma: rozmiar macierzy $n = 1, 2, 3, \ldots, 1000$ (lub maksymalny rozmiar, dla którego udało się wykonać obliczenia).

- Oś pionowa: czas działania rekurencyjnego algorytmu mnożenia macierzy.

5. **Wykres liczby operacji zmiennoprzecinkowych** – 3 punkty.
   - Oś pozioma: rozmiar macierzy $n = 1, 2, 3, \ldots, 1000$ (lub maksymalny rozmiar, dla którego udało się wykonać obliczenia).
   - Oś pionowa: liczba operacji zmiennoprzecinkowych wykonanych przez rekurencyjny algorytm.
6. **Oszacowanie złożoności obliczeniowej** algorytmu na podstawie analizy teoretycznej lub wyników eksperymentalnych – 2 punkty.

2.3. Weryfikacja poprawności wyników

Dla wybranej małej macierzy kwadratowej o rozmiarze co najmniej 4×4 należy:

1. Wykonać mnożenie macierzy w środowisku Octave lub MATLAB.
2. Porównać otrzymany wynik z wynikiem własnej implementacji.
3. Zamieścić w sprawozdaniu wyniki obliczeń oraz raport z porównania.

3. Realizacja ćwiczenia

3.1. Środowisko implementacji

Do realizacji ćwiczenia wybrano język C++ w standardzie C++17.

Macierze reprezentowane są za pomocą kontenera `vector<vector<double>>`, przechowującego wartości zmiennoprzecinkowe typu `double`. Program skompilowano z wykorzystaniem kompilatora `g++` oraz flag optymalizacji `-O2` i `-std=c++17`.

3.2. Generowanie macierzy i zliczanie operacji

Zaimplementowano funkcję `generateMatrix(n)`, generującą dwie losowe macierze kwadratowe A i B o rozmiarze $n \times n$. Elementy macierzy są liczbami typu `double` należącymi do przedziału otwartego $(0.00000001, 1.0)$.

W celu zliczania operacji zmiennoprzecinkowych zastosowano dwa globalne liczniki:
- `g_mults` – liczba wykonanych mnożeń,
- `g_adds` – liczba wykonanych dodawań i odejmowań.

Łączna liczba operacji obliczana jest jako suma `g_mults + g_adds`. Liczniki są zerowane przed uruchomieniem każdego algorytmu.

3.3. Punkt 1 – rekurencyjne mnożenie macierzy metodą Binét'a

Zaimplementowano rekurencyjny algorytm mnożenia macierzy metodą Binét'a w funkcjach `binet()` oraz `binetAdd()`. Algorytm wykorzystuje klasyczny podział macierzy na cztery bloki, a następnie oblicza wynik za pomocą ośmiu rekurencyjnych mnożeń odpowiednich podmacierzy. Jego schemat działania przedstawiono we fragmencie pseudokodu nr 1.

Podział wykonywany jest w połowie każdego wymiaru. Rekurencja kończy działanie, gdy przynajmniej jeden z wymiarów mnożenia jest mniejszy od 2. W takim przypadku stosowany jest klasyczny algorytm mnożenia macierzy.

```
void binetAdd(View A, View B, MView C, bool fresh) {
    int rows = A.rows;
    int inner = A.cols;
    int cols = B.cols;

    if (rows < 2 || inner < 2 || cols < 2) {
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++) {
                double sum = A[i][0] * B[0][j];
                for (int k = 1; k < inner; k++)
                    sum += A[i][k] * B[k][j];
                if (fresh) C[i][j] = sum;
                else       C[i][j] += sum;
            }
        return;
    }

    int mr = rows / 2, mk = inner / 2, mc = cols / 2;
    auto [A11, A12, A21, A22] = split2(A, mr, mk);
    auto [B11, B12, B21, B22] = split2(B, mk, mc);
    auto [C11, C12, C21, C22] = split2(C, mr, mc);

    binetAdd(A11, B11, C11, fresh);
    binetAdd(A12, B21, C11, false);
    binetAdd(A11, B12, C12, fresh);
    binetAdd(A12, B22, C12, false);
    binetAdd(A21, B11, C21, fresh);
    binetAdd(A22, B21, C21, false);
    binetAdd(A21, B12, C22, fresh);
    binetAdd(A22, B22, C22, false);
}

Matrix binet(Matrix A, Matrix B) {
    Matrix C(A.rows, B.cols, 0.0);
    binetAdd(View(A), View(B), MView(C), true);
    return C;
}
```

Fragment pseudokodu 1: Schemat działania metody Binét'a

3.4. Punkt 2 – rekurencyjne mnożenie macierzy metodą Strassena

Zaimplementowano algorytm Strassena w funkcjach `strassen()`, `strassenRec()` oraz `strassen2x2()`. Algorytm dzieli macierze na cztery bloki, a następnie wyznacza siedem iloczynów pośrednich `M1`–`M7`. Na ich podstawie oblicza cztery bloki macierzy wynikowej: C_{11}, C_{12}, C_{21} oraz C_{22}. Jego schemat działania przedstawiono we fragmencie pseudokodu nr 2.

W implementacji zrezygnowano z wcześniejszego dopełniania całych macierzy zerami do rozmiaru będącego potęgą liczby 2. Rozmiar podziału wyznaczany jest na podstawie najbliższej większej lub równej potęgi liczby 2. Dla macierzy o rozmiarze 2×2 zastosowano bezpośredni wariant algorytmu Strassena, wykonujący 7 mnożeń i 18 dodawań lub odejmowań. W pozostałych przypadkach, gdy co najmniej jeden wymiar jest < 2, wykonywane jest klasyczne mnożenie macierzy.

```
Dense strassenRec(View A, View B) {
    if (A.rows == 2 && A.cols == 2 && B.rows == 2 && B.cols == 2)
        return strassen2x2(A, B);  // 7 mnozen, 18 dodawan/odejmowan

    if (A.rows <= 2 || A.cols <= 2 || B.rows <= 2 || B.cols <= 2)
        return brutal(A, B);

    int p = nextPower(maxDimension(A, B), 2);
    int h = p / 2;
    auto [A11, A12, A21, A22] = split2Clipped(A, h, p);
    auto [B11, B12, B21, B22] = split2Clipped(B, h, p);

    Dense M1 = strassenRec(A11 + A22, B11 + B22);
    Dense M2 = strassenRec(A21 + A22, B11);
    Dense M3 = strassenRec(A11, B12 - B22);
    Dense M4 = strassenRec(A22, B21 - B11);
    Dense M5 = strassenRec(A11 + A12, B22);
    Dense M6 = strassenRec(A21 - A11, B11 + B12);
    Dense M7 = strassenRec(A12 - A22, B21 + B22);

    Dense C11 = M1 + M4 - M5 + M7;
    Dense C12 = M3 + M5;
    Dense C21 = M2 + M4;
    Dense C22 = M1 - M2 + M3 + M6;

    Dense C(A.rows, B.cols);
    place(C, C11, 0, 0); place(C, C12, 0, h);
    place(C, C21, h, 0); place(C, C22, h, h);
    return C;
}

Matrix strassen(Matrix A, Matrix B) {
    return strassenRec(View(A), View(B)).d;
}
```

Fragment pseudokodu 2: Schemat działania metody Strassena

3.5. Punkt 3 – rekurencyjne mnożenie macierzy metodą Ladermana

Zaimplementowano rekurencyjny algorytm Ladermana w funkcjach `laderman()` oraz `ladermanRec()`. Algorytm dzieli macierze na dziewięć bloków, a następnie wyznacza 23 iloczyny pośrednie `m1`–`m23`. Na ich podstawie oblicza dziewięć bloków macierzy wynikowej `c11`–`c33`. Jego schemat działania przedstawiono we fragmencie pseudokodu nr 3.

Podobnie jak w implementacji Strassena, nie zastosowano wcześniejszego dopełniania całych macierzy zerami. Wielkość bloków wyznaczana jest na podstawie najbliższej większej lub równej potęgi liczby 3. W pozostałych przypadkach, gdy co najmniej jeden wymiar jest < 3, wykonywane jest klasyczne mnożenie macierzy.

```
Dense ladermanRec(View A, View B) {
    if (A.rows <= 2 || A.cols <= 2 || B.rows <= 2 || B.cols <= 2)
        return brutal(A, B);

    int p = nextPower(maxDimension(A, B), 3);
    int t = p / 3;
    View a[3][3] = split3Clipped(A, t, p);
    View b[3][3] = split3Clipped(B, t, p);

    Dense m1  = ladermanRec(a11+a12+a13-a21-a22-a32-a33, b22);
    Dense m2  = ladermanRec(a11-a21, b22-b12);
    Dense m3  = ladermanRec(a22, b12+b21-b11-b22-b23-b31+b33);
    Dense m4  = ladermanRec(a21+a22-a11, b11-b12+b22);
    Dense m5  = ladermanRec(a21+a22, b12-b11);
    Dense m6  = ladermanRec(a11, b11);
    Dense m7  = ladermanRec(a31+a32-a11, b11-b13+b23);
    Dense m8  = ladermanRec(a31-a11, b13-b23);
    Dense m9  = ladermanRec(a31+a32, b13-b11);
    Dense m10 = ladermanRec(a11+a12+a13-a22-a23-a31-a32, b23);
    Dense m11 = ladermanRec(a32, b13+b21-b11-b22-b23-b31+b32);
    Dense m12 = ladermanRec(a32+a33-a13, b22+b31-b32);
    Dense m13 = ladermanRec(a13-a33, b22-b32);
    Dense m14 = ladermanRec(a13, b31);
    Dense m15 = ladermanRec(a32+a33, b32-b31);
    Dense m16 = ladermanRec(a22+a23-a13, b23+b31-b33);
    Dense m17 = ladermanRec(a13-a23, b23-b33);
    Dense m18 = ladermanRec(a22+a23, b33-b31);
    Dense m19 = ladermanRec(a12, b21);
    Dense m20 = ladermanRec(a23, b32);
    Dense m21 = ladermanRec(a21, b13);
    Dense m22 = ladermanRec(a31, b12);
    Dense m23 = ladermanRec(a33, b33);

    Dense c11 = m6+m14+m19;
    Dense c12 = m1+m4+m5+m6+m12+m14+m15;
    Dense c13 = m6+m7+m9+m10+m14+m16+m18;
    Dense c21 = m2+m3+m4+m6+m14+m16+m17;
    Dense c22 = m2+m4+m5+m6+m20;
    Dense c23 = m14+m16+m17+m18+m21;
    Dense c31 = m6+m7+m8+m11+m12+m13+m14;
    Dense c32 = m12+m13+m14+m15+m22;
    Dense c33 = m6+m7+m8+m9+m23;
```

```
43
44      Dense C(A.rows, B.cols);
45      place(C, c11, 0, 0);   place(C, c12, 0, t);   place(C, c13, 0, 2*t);
46      place(C, c21, t, 0);   place(C, c22, t, t);   place(C, c23, t, 2*t);
47      place(C, c31, 2*t, 0); place(C, c32, 2*t, t); place(C, c33, 2*t, 2*t);
48      return C;
49  }
50
51  Matrix laderman(Matrix A, Matrix B) {
52      return ladermanRec(View(A), View(B)).d;
53  }
```

Fragment pseudokodu 3: Schemat działania metody Ladermana

3.6. Pomiar czasu i weryfikacja poprawności

Funkcja `runTest()` uruchamia wybraną metodę mnożenia macierzy. Pomiar czasu realizowany jest za pomocą `chrono::steady_clock` [ms]. Klasyczne mnożenie macierzy w funkcji `classical()`, służy jako wynik referencyjny.

Dla macierzy o rozmiarze nieprzekraczającym 10×10 program wyświetla macierze wejściowe, wynik oraz statystyki operacji.

Wszystkie cztery metody – klasyczna, Binét'a, Strassena i Ladermana – zaimplementowano w pliku `main.cpp` fragment kodu 4).

```
1  #include <iostream>
2  #include <vector>
3  #include <cstdlib>
4  #include <ctime>
5  #include <cmath>
6  #include <iomanip>
7  #include <utility>
8  #include <algorithm>
9  #include <initializer_list>
10 #include <string>
11 #include <sstream>
12 #include <type_traits>
13 #include <chrono>
14
15 using namespace std;
16 using Matrix = vector<vector<double>>;
17
18 template <typename T>
19 void draw(const vector<vector<T>> &M, int precision) {
20     if (M.empty()) {
21         cout << "[ ]\n";
22         return;
23     }
24
25     size_t rows = M.size();
26     size_t cols = 0;
27     for (const auto &row : M) cols = max(cols, row.size());
28
29     vector<vector<string>> cells(rows, vector<string>(cols));
30     vector<size_t> width(cols, 0);
31
```

```
    for (size_t r = 0; r < rows; ++r) {
        for (size_t c = 0; c < M[r].size(); ++c) {
            ostringstream ss;
            if constexpr (is_floating_point_v<T>)
                ss << fixed << setprecision(precision);
            ss << M[r][c];
            cells[r][c] = ss.str();
            width[c] = max(width[c], cells[r][c].size());
        }
    }

    for (size_t r = 0; r < rows; ++r) {
        string left, right;
        if (rows == 1)          { left = "[";  right = "]"; }
        else if (r == 0)        { left = "┌";  right = "┐"; }
        else if (r == rows - 1) { left = "└";  right = "┘"; }
        else                    { left = "│";  right = "│"; }

        cout << left;
        for (size_t c = 0; c < cols; ++c) {
            cout << " " << setw(width[c]) << cells[r][c];
        }
        cout << " " << right << "\n";
    }
}

static unsigned long long g_mults = 0;
static unsigned long long g_adds  = 0;

const int MAX_DRAW_SIZE = 10;
const int PRINT_PRECISION = 4;

Matrix generateMatrix(int n) {
    const double LO = 1e-8;
    Matrix M(n, vector<double>(n));
    for (auto &row : M)
        for (auto &x : row) {
            double u = (rand() + 1.0) / (RAND_MAX + 2.0);
            x = LO + (1.0 - LO) * u;
        }
    return M;
}

struct Dense {
    int r = 0, c = 0;
    Matrix d;
    Dense() {}
    Dense(int rows, int cols) : r(rows), c(cols), d(rows, vector<double>(cols, 0.0))
{}
};

struct View {
    const Matrix *m;
    int r0, c0, rows, cols;
    View(const Matrix *mm, int row0, int col0, int nr, int nc) : m(mm), r0(row0),
c0(col0), rows(nr), cols(nc) {}
    View(const Matrix &M) : m(&M), r0(0), c0(0), rows(M.size()), cols(M.empty() ? 0 :
M[0].size()) {}
    View(const Dense &x) : m(&x.d), r0(0), c0(0), rows(x.r), cols(x.c) {}
    double at(int i, int j) const { return (*m)[r0 + i][c0 + j]; }
};
```

```

struct MView {
    Matrix *m;
    int r0, c0, rows, cols;
};

View block(const View &v, int ra, int rb, int ca, int cb) {
    ra = min(ra, v.rows); rb = min(rb, v.rows);
    ca = min(ca, v.cols); cb = min(cb, v.cols);
    return View(v.m, v.r0 + ra, v.c0 + ca, rb - ra, cb - ca);
}

MView block(const MView &v, int ra, int rb, int ca, int cb) {
    return MView{v.m, v.r0 + ra, v.c0 + ca, rb - ra, cb - ca};
}

int nextPower(int x, int base) {
    int p = 1;
    while (p < x) p *= base;
    return p;
}

Dense combination(initializer_list<pair<int, View>> terms) {
    int R = 0, C = 0;
    for (const auto &t : terms) { R = max(R, t.second.rows); C = max(C,
t.second.cols); }
    Dense res(R, C);
    vector<int> covered(R, 0);
    unsigned long long ops = 0;
    for (const auto &t : terms) {
        const View &v = t.second;
        for (int i = 0; i < v.rows; i++) {
            for (int j = 0; j < v.cols; j++) {
                double x = t.first * v.at(i, j);
                if (j < covered[i]) { res.d[i][j] += x; ops++; }
                else res.d[i][j] = x;
            }
            covered[i] = max(covered[i], v.cols);
        }
    }
    g_adds += ops;
    return res;
}

void place(Dense &C, const Dense &X, int ro, int co) {
    for (int i = 0; i < X.r && ro + i < C.r; i++)
        for (int j = 0; j < X.c && co + j < C.c; j++) C.d[ro + i][co + j] = X.d[i][j];
}

Dense brutal(const View &a, const View &b) {
    int r = a.rows, c = b.cols, k = min(a.cols, b.rows);
    Dense res(r, c);
    if (k == 0) return res;
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) {
            double s = a.at(i, 0) * b.at(0, j);
            for (int l = 1; l < k; l++) s += a.at(i, l) * b.at(l, j);
            res.d[i][j] = s;
        }
    g_mults += (unsigned long long)r * k * c;
    g_adds  += (unsigned long long)r * (k - 1) * c;
```

```
    return res;
}

Matrix classical(const Matrix &A, const Matrix &B) {
    int n = A.size();
    Matrix C(n, vector<double>(n, 0.0));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++)
            for (int j = 0; j < n; j++) C[i][j] += A[i][k] * B[k][j];
    unsigned long long N = n;
    g_mults += N * N * N;
    g_adds  += N * N * (N - 1);
    return C;
}

void binetAdd(const View &a, const View &b, const MView &res, bool fresh) {
    int n1 = a.rows, n2 = b.rows, m2 = b.cols;

    if (n1 < 2 || n2 < 2 || m2 < 2) {
        int k = a.cols;
        for (int i = 0; i < n1; i++)
            for (int j = 0; j < m2; j++) {
                double s = a.at(i, 0) * b.at(0, j);
                for (int l = 1; l < k; l++) s += a.at(i, l) * b.at(l, j);
                double &target = (*res.m)[res.r0 + i][res.c0 + j];
                if (fresh) target = s; else target += s;
            }
        g_mults += (unsigned long long)n1 * k * m2;
        g_adds  += (unsigned long long)n1 * (fresh ? k - 1 : k) * m2;
        return;
    }

    int mr = n1 / 2, mk = n2 / 2, mc = m2 / 2;
    View a11 = block(a, 0, mr, 0, mk),  a12 = block(a, 0, mr, mk, a.cols);
    View a21 = block(a, mr, n1, 0, mk), a22 = block(a, mr, n1, mk, a.cols);
    View b11 = block(b, 0, mk, 0, mc),  b12 = block(b, 0, mk, mc, m2);
    View b21 = block(b, mk, n2, 0, mc), b22 = block(b, mk, n2, mc, m2);
    MView r11 = block(res, 0, mr, 0, mc),  r12 = block(res, 0, mr, mc, m2);
    MView r21 = block(res, mr, n1, 0, mc), r22 = block(res, mr, n1, mc, m2);

    binetAdd(a11, b11, r11, fresh); binetAdd(a12, b21, r11, false);
    binetAdd(a11, b12, r12, fresh); binetAdd(a12, b22, r12, false);
    binetAdd(a21, b11, r21, fresh); binetAdd(a22, b21, r21, false);
    binetAdd(a21, b12, r22, fresh); binetAdd(a22, b22, r22, false);
}

Matrix binet(const Matrix &A, const Matrix &B) {
    int n = A.size();
    Matrix C(n, vector<double>(B[0].size(), 0.0));
    binetAdd(View(A), View(B), MView{&C, 0, 0, n, (int)B[0].size()}, true);
    return C;
}


Dense strassen2x2(const View &a, const View &b) {
    double a11 = a.at(0, 0), a12 = a.at(0, 1), a21 = a.at(1, 0), a22 = a.at(1, 1);
    double b11 = b.at(0, 0), b12 = b.at(0, 1), b21 = b.at(1, 0), b22 = b.at(1, 1);
    double m1 = (a11 + a22) * (b11 + b22);
    double m2 = (a21 + a22) * b11;
    double m3 = a11 * (b12 - b22);
    double m4 = a22 * (b21 - b11);
```

```
    double m5 = (a11 + a12) * b22;
    double m6 = (a21 - a11) * (b11 + b12);
    double m7 = (a12 - a22) * (b21 + b22);
    Dense C(2, 2);
    C.d[0][0] = m1 + m4 - m5 + m7;
    C.d[0][1] = m3 + m5;
    C.d[1][0] = m2 + m4;
    C.d[1][1] = m1 - m2 + m3 + m6;
    g_mults += 7;
    g_adds  += 18;
    return C;
}

Dense strassenRec(const View &a, const View &b) {
    int r1 = a.rows, c1 = a.cols, r2 = b.rows, c2 = b.cols;
    if (r1 == 2 && c1 == 2 && r2 == 2 && c2 == 2) return strassen2x2(a, b);
    if (r1 <= 2 || c1 <= 2 || r2 <= 2 || c2 <= 2) return brutal(a, b);

    int mx = max(max(r1, c1), max(r2, c2));
    int p = nextPower(mx, 2);
    int h = p / 2;

    View a11 = block(a, 0, h, 0, h),     a12 = block(a, 0, h, h, p);
    View a21 = block(a, h, p, 0, h),     a22 = block(a, h, p, h, p);
    View b11 = block(b, 0, h, 0, h),     b12 = block(b, 0, h, h, p);
    View b21 = block(b, h, p, 0, h),     b22 = block(b, h, p, h, p);

    Dense M1 = strassenRec(combination({{1, a11}, {1, a22}}), combination({{1, b11},
{1, b22}}));
    Dense M2 = strassenRec(combination({{1, a21}, {1, a22}}), b11);
    Dense M3 = strassenRec(a11, combination({{1, b12}, {-1, b22}}));
    Dense M4 = strassenRec(a22, combination({{1, b21}, {-1, b11}}));
    Dense M5 = strassenRec(combination({{1, a11}, {1, a12}}), b22);
    Dense M6 = strassenRec(combination({{1, a21}, {-1, a11}}), combination({{1, b11},
{1, b12}}));
    Dense M7 = strassenRec(combination({{1, a12}, {-1, a22}}), combination({{1, b21},
{1, b22}}));

    Dense C11 = combination({{1, M1}, {1, M4}, {-1, M5}, {1, M7}});
    Dense C12 = combination({{1, M3}, {1, M5}});
    Dense C21 = combination({{1, M2}, {1, M4}});
    Dense C22 = combination({{1, M1}, {-1, M2}, {1, M3}, {1, M6}});

    Dense C(r1, c2);
    place(C, C11, 0, 0); place(C, C12, 0, h);
    place(C, C21, h, 0); place(C, C22, h, h);
    return C;
}

Matrix strassen(const Matrix &A, const Matrix &B) {
    return strassenRec(View(A), View(B)).d;
}

Dense ladermanRec(const View &A, const View &B) {
    int r1 = A.rows, c1 = A.cols, r2 = B.rows, c2 = B.cols;
    if (r1 <= 2 || c1 <= 2 || r2 <= 2 || c2 <= 2) return brutal(A, B);

    int mx = max(max(r1, c1), max(r2, c2));
    int p = nextPower(mx, 3);
    int t = p / 3;

```

```
    View a[3][3] = {{block(A, 0, t, 0, t),     block(A, 0, t, t, 2 * t),     block(A, 0, t, 2 * t, p)},
                    {block(A, t, 2 * t, 0, t), block(A, t, 2 * t, t, 2 * t), block(A, t, 2 * t, 2 * t, p)},
                    {block(A, 2 * t, p, 0, t), block(A, 2 * t, p, t, 2 * t), block(A, 2 * t, p, 2 * t, p)}};
    View b[3][3] = {{block(B, 0, t, 0, t),     block(B, 0, t, t, 2 * t),     block(B, 0, t, 2 * t, p)},
                    {block(B, t, 2 * t, 0, t), block(B, t, 2 * t, t, 2 * t), block(B, t, 2 * t, 2 * t, p)},
                    {block(B, 2 * t, p, 0, t), block(B, 2 * t, p, t, 2 * t), block(B, 2 * t, p, 2 * t, p)}};
    const View &a11 = a[0][0], &a12 = a[0][1], &a13 = a[0][2];
    const View &a21 = a[1][0], &a22 = a[1][1], &a23 = a[1][2];
    const View &a31 = a[2][0], &a32 = a[2][1], &a33 = a[2][2];
    const View &b11 = b[0][0], &b12 = b[0][1], &b13 = b[0][2];
    const View &b21 = b[1][0], &b22 = b[1][1], &b23 = b[1][2];
    const View &b31 = b[2][0], &b32 = b[2][1], &b33 = b[2][2];

    auto mul = ladermanRec;
    Dense m1  = mul(combination({{1,a11},{1,a12},{1,a13},{-1,a21},{-1,a22},{-1,a32},{-1,a33}}), b22);
    Dense m2  = mul(combination({{1,a11},{-1,a21}}), combination({{1,b22},{-1,b12}}));
    Dense m3  = mul(a22, combination({{1,b12},{1,b21},{-1,b11},{-1,b22},{-1,b23},{-1,b31},{1,b33}}));
    Dense m4  = mul(combination({{1,a21},{1,a22},{-1,a11}}), combination({{1,b11},{-1,b12},{1,b22}}));
    Dense m5  = mul(combination({{1,a21},{1,a22}}), combination({{1,b12},{-1,b11}}));
    Dense m6  = mul(a11, b11);
    Dense m7  = mul(combination({{1,a31},{1,a32},{-1,a11}}), combination({{1,b11},{-1,b13},{1,b23}}));
    Dense m8  = mul(combination({{1,a31},{-1,a11}}), combination({{1,b13},{-1,b23}}));
    Dense m9  = mul(combination({{1,a31},{1,a32}}), combination({{1,b13},{-1,b11}}));
    Dense m10 = mul(combination({{1,a11},{1,a12},{1,a13},{-1,a22},{-1,a23},{-1,a31},{-1,a32}}), b23);
    Dense m11 = mul(a32, combination({{1,b13},{1,b21},{-1,b11},{-1,b22},{-1,b23},{-1,b31},{1,b32}}));
    Dense m12 = mul(combination({{1,a32},{1,a33},{-1,a13}}), combination({{1,b22},{1,b31},{-1,b32}}));
    Dense m13 = mul(combination({{1,a13},{-1,a33}}), combination({{1,b22},{-1,b32}}));
    Dense m14 = mul(a13, b31);
    Dense m15 = mul(combination({{1,a32},{1,a33}}), combination({{1,b32},{-1,b31}}));
    Dense m16 = mul(combination({{1,a22},{1,a23},{-1,a13}}), combination({{1,b23},{1,b31},{-1,b33}}));
    Dense m17 = mul(combination({{1,a13},{-1,a23}}), combination({{1,b23},{-1,b33}}));
    Dense m18 = mul(combination({{1,a22},{1,a23}}), combination({{1,b33},{-1,b31}}));
    Dense m19 = mul(a12, b21);
    Dense m20 = mul(a23, b32);
    Dense m21 = mul(a21, b13);
    Dense m22 = mul(a31, b12);
    Dense m23 = mul(a33, b33);

    Dense c11 = combination({{1,m6},{1,m14},{1,m19}});
    Dense c12 = combination({{1,m1},{1,m4},{1,m5},{1,m6},{1,m12},{1,m14},{1,m15}});
    Dense c13 = combination({{1,m6},{1,m7},{1,m9},{1,m10},{1,m14},{1,m16},{1,m18}});
    Dense c21 = combination({{1,m2},{1,m3},{1,m4},{1,m6},{1,m14},{1,m16},{1,m17}});
    Dense c22 = combination({{1,m2},{1,m4},{1,m5},{1,m6},{1,m20}});
    Dense c23 = combination({{1,m14},{1,m16},{1,m17},{1,m18},{1,m21}});
    Dense c31 = combination({{1,m6},{1,m7},{1,m8},{1,m11},{1,m12},{1,m13},{1,m14}});
    Dense c32 = combination({{1,m12},{1,m13},{1,m14},{1,m15},{1,m22}});
    Dense c33 = combination({{1,m6},{1,m7},{1,m8},{1,m9},{1,m23}});
```

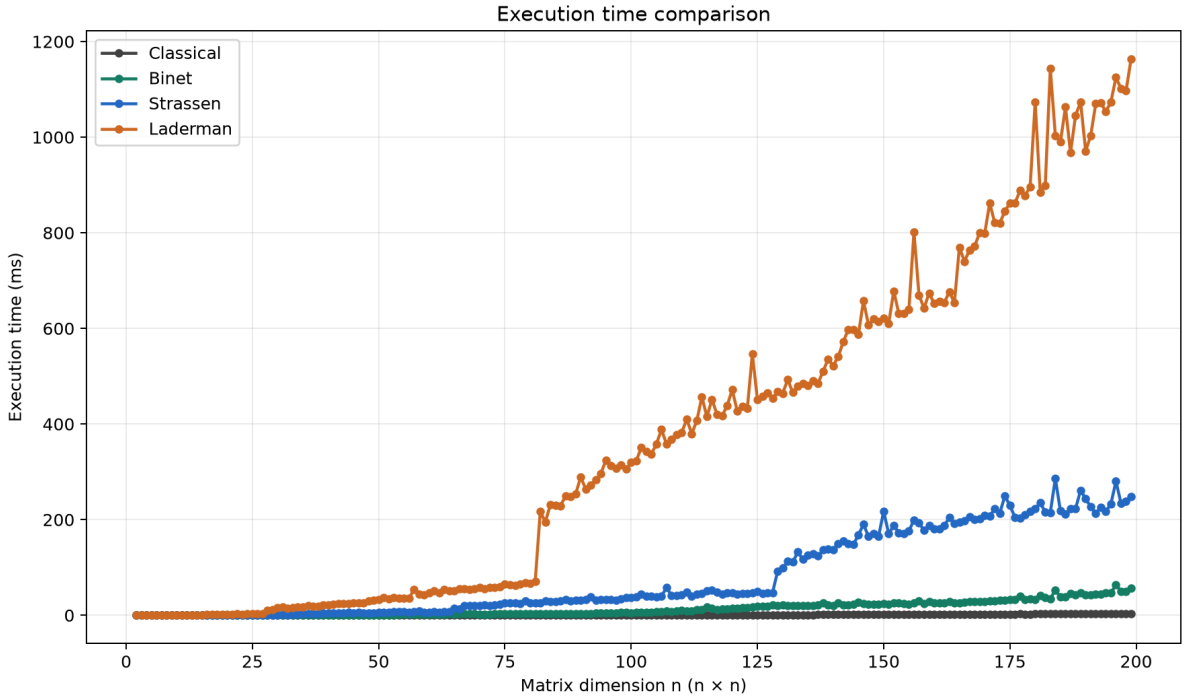
```
316
317     Dense C(r1, c2);
318     place(C, c11, 0, 0);      place(C, c12, 0, t);      place(C, c13, 0, 2 * t);
319     place(C, c21, t, 0);      place(C, c22, t, t);      place(C, c23, t, 2 * t);
320     place(C, c31, 2 * t, 0);  place(C, c32, 2 * t, t);  place(C, c33, 2 * t, 2 * t);
321     return C;
322 }
323
324 Matrix laderman(const Matrix &A, const Matrix &B) {
325     return ladermanRec(View(A), View(B)).d;
326 }
327
328 double maxDifference(const Matrix &A, const Matrix &B) {
329     double m = 0;
330     for (size_t i = 0; i < A.size(); i++)
331         for (size_t j = 0; j < A[i].size(); j++) m = max(m, fabs(A[i][j] - B[i][j]));
332     return m;
333 }
334
335 void runTest(const string &name, Matrix (*f)(const Matrix &, const Matrix &),
336              const Matrix &A, const Matrix &B, const Matrix &ref, bool verbose) {
337     int n = A.size();
338     g_mults = 0;
339     g_adds = 0;
340
341     auto start = chrono::steady_clock::now();
342     Matrix C = f(A, B);
343     auto stop = chrono::steady_clock::now();
344     double ms = chrono::duration<double, milli>(stop - start).count();
345
346     cout << left << setw(10) << name << " | time: " << fixed << setprecision(3)
347          << setw(12) << ms << " ms";
348
349     if (!verbose) {
350         cout << "\n";
351         return;
352     }
353
354     cout << " | mults: " << setw(10) << g_mults
355          << " | adds: " << setw(10) << g_adds
356          << " | total: " << setw(10) << g_mults + g_adds
357          << " | max |error|: " << scientific << setprecision(3) << maxDifference(C,
ref)
358          << fixed << "\n";
359     if (n <= MAX_DRAW_SIZE) {
360         draw(C, PRINT_PRECISION);
361         cout << "\n";
362     }
363 }
364
365 int main() {
366     const bool verbose = true;
367
368     srand(time(nullptr));
369     cout << "Enter matrix size n: ";
370     int n;
371     if (!(cin >> n) || n < 1) { cout << "Invalid size\n"; return 1; }
372
373     Matrix A = generateMatrix(n);
374     Matrix B = generateMatrix(n);
375     Matrix ref;
```

```
376     if (verbose) {
377         ref = classical(A, B);
378         if (n <= MAX_DRAW_SIZE) {
379             cout << "\nA =\n";   draw(A, PRINT_PRECISION);
380             cout << "\nB =\n";   draw(B, PRINT_PRECISION);
381             cout << "\nA * B (classical) =\n"; draw(ref, PRINT_PRECISION);
382         }
383     }
384 
385     cout << "\nn = " << n << "\n\n";
386     runTest("Classical", classical, A, B, ref, verbose);
387     runTest("Binet",    binet,    A, B, ref, verbose);
388     runTest("Strassen", strassen, A, B, ref, verbose);
389     runTest("Laderman", laderman, A, B, ref, verbose);
390 
391     return 0;
392 }
```

Fragment kodu 4: Plik `main.cpp` – metoda klasyczna, Binet'a, Strassena oraz Ladermana

4. Wyniki

4.1. Czas działania

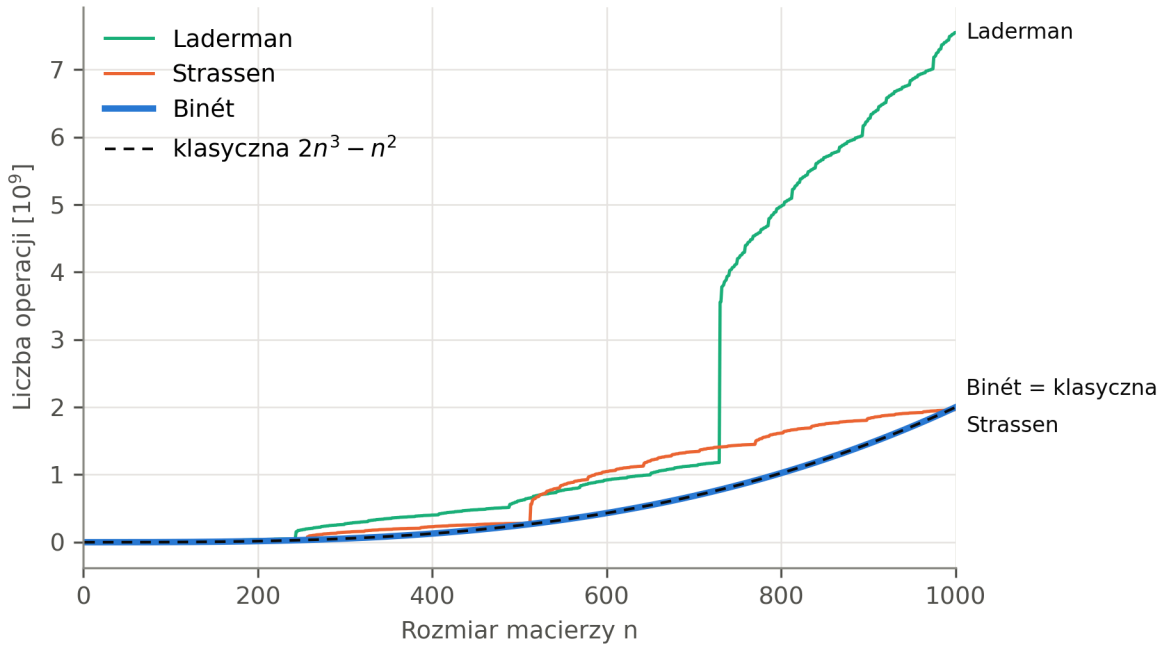


Rysunek 1: Czas działania metod w zależności od rozmiaru macierzy n

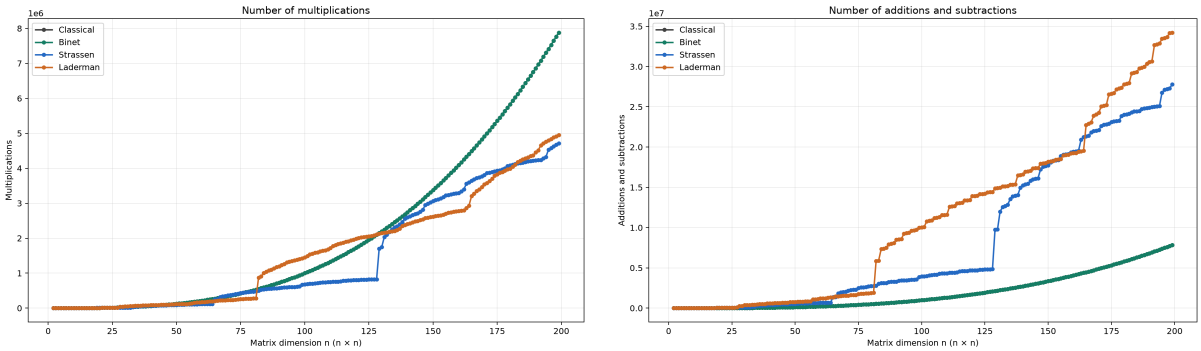
Najszybsza metodą okazała się metoda Binét'a (rysunek 1). Liczy ona wynik w miejscu, na „widokach" podmacierzy, bez alokowania nowej pamięci. Metody Strassena i Ladermana są wielokrotnie wolniejsze: każda suma bloków w tworzy nową macierz, a rekurencja schodzi do bloków 2×2, więc czas zdominowany jest przez alokacje pamięci i dodawania. Najwolniejsza jest metoda Ladermana (23 wywołania rekurencyjne i 98 dodawań bloków na poziom, wobec 7 i 18 u Strassena). Skoki czasu po przekroczeniu potęgi 2 (Strassen, $n = 512 \to 513$) i potęgi 3 (Laderman, $n = 243 \to 244$) wyjaśniono w kolejnym podrozdziale.

4.2. Liczba operacji zmiennoprzecinkowych

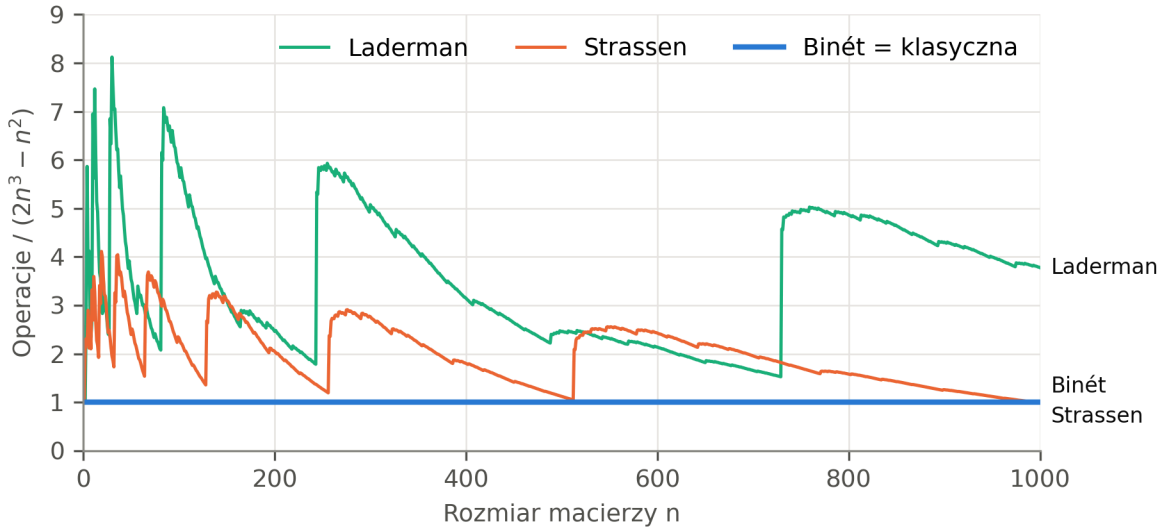
Liczba operacji zależy tylko od n, dlatego wyznaczono ją dla każdego $n = 1, \ldots, 1000$ tą samą logiką zliczania co w programie (dla wybranych n wyniki pokrywają się dokładnie z licznikami `g_mults` i `g_adds`). Liczone są tylko działania na dwóch rzeczywistych liczbach.



Rysunek 2: Liczba operacji zmiennoprzecinkowych: linia przerywana – metoda klasyczna, $2n^3 - n^2$



Rysunek 3: Liczba mnożeń (po lewej) oraz dodawań i odejmowań (po prawej)



Rysunek 4: Liczba operacji względem metody klasycznej $2n^3 - n^2$

- **Binét** wykonuje dokładnie $2n^3 - n^2$ operacji, czyli tyle co metoda klasyczna – rekurencja zmienia tylko kolejność obliczania tych samych iloczynów, dlatego krzywe na rysunku 2 się pokrywają.
- **Strassen** wykonuje najmniej mnożeń (dla $n = 1000$ – 28% mnożeń metody klasycznej), ale dodawania stanowią ok. 86% jego operacji, więc łącznie wyprzedza metodę klasyczną dopiero dla $n \geq 993$.
- **Laderman** dla każdego $n \geq 3$ wykonuje więcej operacji niż metoda klasyczna (np. 1,52 razy więcej dla $n = 729$ i 3,78 razy dla $n = 1000$). Dodawania stanowią ok. 87% jego operacji.

Skoki tuż po potęgach liczby 2 (Strassen: $n = 129, 257, 513$, ok. 2 razy) i 3 (Laderman: $n = 244, 730$, ok. 3 razy) wynikają z podziału bez dopełniania zerami. Dla $n = 513$ bloki mają rozmiary 512 i 1, więc tylko 2 z 7 iloczynów (M_1, M_6) działają na pełnych blokach 512×512. Dla $n = 730$ bloki mają rozmiary 729, 1 i 0, a pełny rozmiar mają 3 z 23 iloczynów (m_4, m_6, m_7).

n	Binét = klasyczna	Strassen	Strassen / klasyczna	Laderman	Laderman / klasyczna
100	$1{,}99 \cdot 10^6$	$4{,}63 \cdot 10^6$	2,33	$1{,}15 \cdot 10^7$	5,76
512	$2{,}68 \cdot 10^8$	$2{,}81 \cdot 10^8$	1,05	$6{,}54 \cdot 10^8$	2,44
513	$2{,}70 \cdot 10^8$	$5{,}64 \cdot 10^8$	2,09	$6{,}58 \cdot 10^8$	2,44
729	$7{,}74 \cdot 10^8$	$1{,}41 \cdot 10^9$	1,82	$1{,}18 \cdot 10^9$	1,52
730	$7{,}78 \cdot 10^8$	$1{,}41 \cdot 10^9$	1,81	$3{,}56 \cdot 10^9$	4,57
1000	$2{,}00 \cdot 10^9$	$1{,}96 \cdot 10^9$	0,98	$7{,}55 \cdot 10^9$	3,78

Tabela 1: Liczba operacji zmiennoprzecinkowych dla wybranych rozmiarów macierzy

5. Złożoność obliczeniowa

5.1. Analiza teoretyczna

Każda metoda dzieli macierz na bloki o boku $\frac{n}{b}$ i wykonuje a rekurencyjnych mnożeń oraz c dodawań bloków, więc

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + c \cdot \left(\frac{n}{b}\right)^2.$$

Ponieważ $\log_b a > 2$, z twierdzenia o rekurencji uniwersalnej (*master theorem*) $T(n) = \Theta\left(n^{\log_b a}\right)$ – złożoność wyznacza liczba mnożeń rekurencyjnych.

Metoda	a	b	c	Złożoność
Binét	8	2	4	$\Theta(n^3)$
Strassen	7	2	18	$\Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2{,}807})$
Laderman	23	3	98	$\Theta\left(n^{\log_3 23}\right) \approx \Theta(n^{2{,}854})$

Tabela 2: Parametry równań rekurencyjnych i wynikająca z nich złożoność

Dla potęg podziału rekurencję można rozwiązać dokładnie (wzory zgadzają się z licznikami programu dla wszystkich takich $n \leq 1000$):

- Binét (dowolne n): $T(n) = 2n^3 - n^2$,
- Strassen ($n = 2^k$): $T(n) = 7n^{\log_2 7} - 6n^2$,
- Laderman ($n = 3^k$): $T(n) = 8n^{\log_3 23} - 7n^2$.

Dla pozostałych n liczba operacji leży między wartościami dla sąsiednich potęg, więc zmienia się tylko stała, a nie klasa złożoności. Z wzorów wynika też, że metoda Strassena wykonuje mniej operacji niż klasyczna dopiero od $n = 2^{10} = 1024$, a metoda Ladermana od $n = 3^9 = 19683$ – stąd wyniki w badanym zakresie $n \leq 1000$.

5.2. Weryfikacja eksperymentalna

Wykładnik złożoności oszacowano z liczby operacji (czas jest zaburzony narzutem alokacji) jako nachylenie wykresu w skali logarytmicznej między kolejnymi potęgami podziału:

$$\alpha = \frac{\ln T(n_2) - \ln T(n_1)}{\ln n_2 - \ln n_1}.$$

Metoda	Przedział n	α	Teoria
Binét	10 – 20	3,037	3
	500 – 1000	3,001	
Strassen	8 – 16	2,921	2,807
	256 – 512	2,813	
Laderman	9 – 27	2,936	2,854
	243 – 729	2,859	

Tabela 3: Eksperymentalny wykładnik złożoności α

Wykładnik maleje wraz ze wzrostem n i zbliża się do wartości teoretycznej (przy małych n zawyża go składnik z n^2), co potwierdza złożoność $\Theta(n^3)$ dla metody Binét'a, $\Theta(n^{2{,}807})$ dla metody Strassena i $\Theta(n^{2{,}854})$ dla metody Ladermana.

6. Wnioski

Wszystkie trzy algorytmy dają poprawne wyniki, a ich złożoność obliczeniowa została potwierdzona eksperymentalnie. Wykładniki wyznaczone z liczby operacji zbliżają się do wartości teoretycznych: $\Theta(n^3)$ dla metody Binét'a, $\Theta(n^{2,807})$ dla metody Strassena i $\Theta(n^{2,854})$ dla metody Ladermana.

Metoda Binét'a wykonuje dokładnie tyle samo operacji co metoda klasyczna, ponieważ rekurencja zmienia jedynie kolejność obliczania tych samych iloczynów. Strassen i Laderman zmniejszają liczbę mnożeń, ale kosztem znacznie większej liczby dodawań.

W praktyce najszybsza okazała się metoda Binét'a, ze względu na wykonywanie obliczeń w miejscu. Metody Strassena i Ladermana są dużo wolniejsze. Ich czas wykonania jest zwiększony przez rekurencyjną alokację pamięci dla nowych podmacierzy. Zastosowanie tych algorytmów ma sens po optymalizacji lub dla dużo większych macierzy.